

Informatik: Algorithmen und Big O - Jahr 10

Grundlagen von Algorithmen und Komplexitätsanalyse

Ein Algorithmus ist eine Schritt-für-Schritt-Anleitung zur Lösung eines Problems.

Eigenschaften guter Algorithmen:

1. Korrektheit: Löst das Problem richtig
2. Effizienz: Verwendet minimale Ressourcen
3. Verständlichkeit: Ist leicht zu verstehen
4. Terminierung: Endet nach endlicher Zeit

Big O Notation:

Big O beschreibt die Leistung eines Algorithmus bei großen Eingabemengen.

Häufige Komplexitätsklassen:

1. $O(1)$ - Konstante Zeit:

- Unabhängig von der Eingabegröße
- Beispiel: Array-Zugriff

2. $O(\log n)$ - Logarithmisch:

- Beispiel: Binäre Suche

3. $O(n)$ - Linear:

- Proportional zur Eingabegröße
- Beispiel: Lineare Suche

4. $O(n^2)$ - Quadratisch:

- Doppelte Schleife
- Beispiel: Bubble Sort

5. $O(n^3)$ - Kubisch:

- Dreifache Schleife

6. $O(2^n)$ - Exponentiell:

- Sehr ineffizient

7. $O(n!)$ - Faktoriell:

- Äußerst ineffizient

Beispiel-Algorithmen:

Lineare Suche:

...

for $i = 0$ to $n-1$:

if $\text{array}[i] == \text{target}$:

return i

return -1

...

Komplexität: $O(n)$

Binäre Suche:

...

left = 0, right = $n-1$

while left <= right:

mid = (left + right) / 2

if $\text{array}[\text{mid}] == \text{target}$: return mid

if $\text{array}[\text{mid}] < \text{target}$: left = mid + 1

else: right = mid - 1

...

Komplexität: $O(\log n)$

Wichtige Erkenntnisse:

- Algorithmuseffizienz wird kritisch bei großen Datenmengen
- Wahl des richtigen Algorithmus ist entscheidend
- Raumkomplexität ist ebenso wichtig wie Zeitkomplexität